Firestore in GCP

What is Firestore?

Firestore, also known as **Cloud Firestore**, is a **fully managed, serverless NoSQL document database** offered by Google Cloud Platform (GCP). It is part of the Firebase and Google Cloud ecosystems and is designed to store, sync, and query data for web, mobile, and server applications at global scale. Firestore is the next-generation version of Firebase Realtime Database, offering more advanced querying and scalability features.

Firestore stores data in **documents**, which are organized into **collections**. Each document contains a set of key-value pairs and can contain nested data structures. It supports real-time data synchronization and offline capabilities, making it suitable for a variety of application types.

Use Cases

1. **Real-Time Applications**
    - Chat applications, live dashboards, and collaborative platforms benefit from Firestore's real-time synchronization features, allowing multiple users to see updates instantly.
2. **Mobile and Web Applications**
    - Ideal for mobile and web apps due to its native integration with Firebase SDKs, offline support, and seamless synchronization.
3. **Content Management Systems (CMS)**
    - Firestore can be used to store structured content for websites or mobile apps, enabling dynamic updates without redeploying code.
4. **E-commerce Platforms**
    - Used for managing product catalogs, user profiles, carts, and order histories, with support for complex queries and transactions.
5. **IoT Applications**
    - Firestore can store sensor data or device states and provide real-time updates to dashboards or control systems.
6. **Gaming Backends**
    - Useful for storing player profiles, leaderboards, and game state with real-time updates.

Benefits

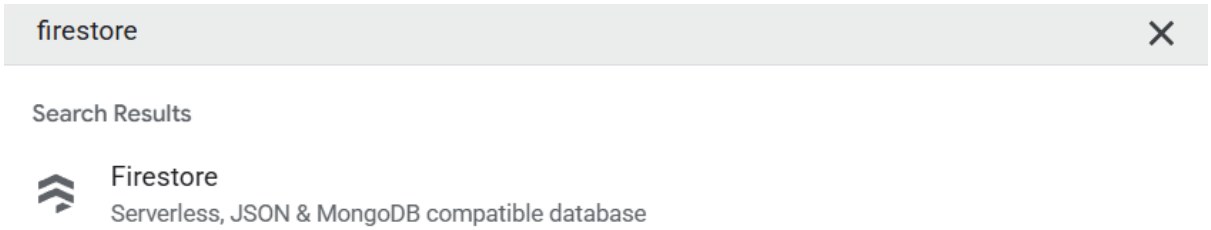
1. **Real-Time Synchronization**
    - Automatically syncs data across clients in real time, improving the user experience in collaborative and dynamic applications.
2. **Offline Support**
    - Allows mobile and web applications to continue functioning even when offline. Data is automatically synced once the connection is reestablished.
3. **Scalability**
    - Firestore is designed to scale horizontally, handling millions of users and large datasets without manual intervention.
4. **Serverless and Fully Managed**
    - Eliminates the need for infrastructure management, automatic scaling, and maintenance operations like patching and backups.
5. **Strong Security**
    - Offers fine-grained security through Firebase Authentication and Firestore Security Rules, enabling role-based access control.
6. **Flexible Data Model**
    - Supports nested objects, arrays, and a variety of data types. Its document-oriented structure is well-suited for hierarchical and structured data.
7. **Integrated with Google Cloud Ecosystem**
    - Easily integrates with other GCP services such as Cloud Functions, Cloud Storage, and BigQuery, enhancing analytics and backend capabilities.
8. **Multi-region and High Availability**
    - Provides strong consistency and replication across multiple regions, improving fault tolerance and availability.

Conclusion

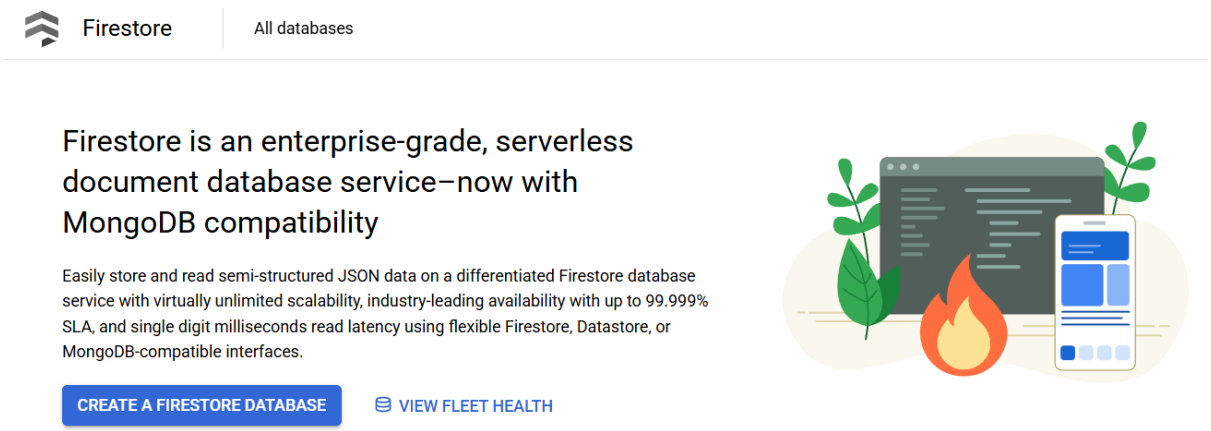
Firestore is a robust, scalable, and developer-friendly NoSQL database service on GCP that supports a wide range of real-time, mobile, web, and server-side application use cases. Its real-time capabilities, offline support, and deep integration with Firebase and Google Cloud make it a powerful tool for modern application development.

To begin with the Lab

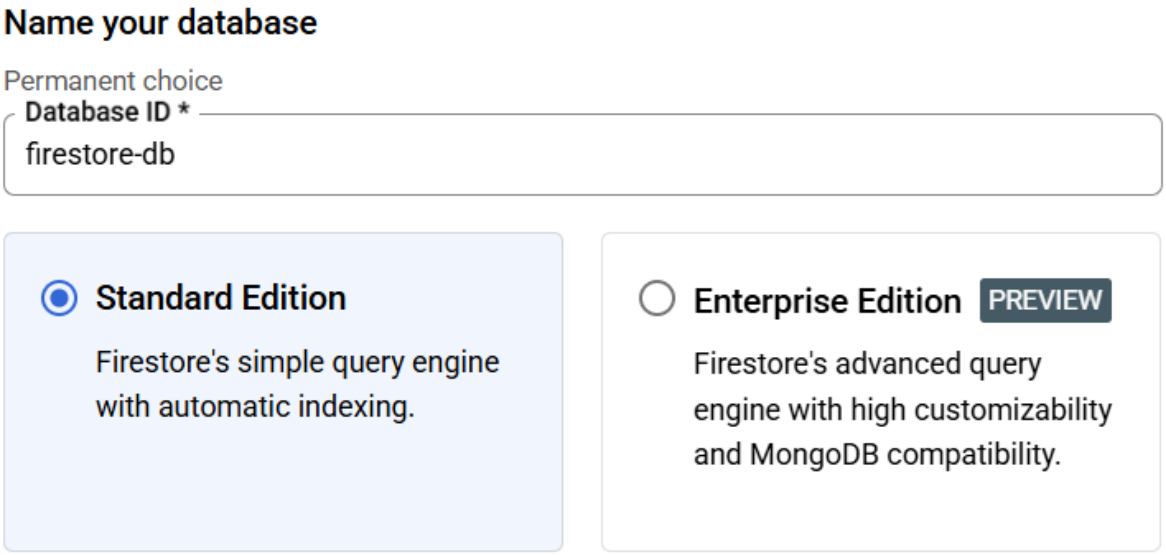
1. In the Google Cloud Console, search for Firestore and choose the service accordingly.



2. Then, from the database of Firestore, click on the Firestore database.



3. Give a name to your database and choose Standard Edition.



4. For the configuration options, choose Firestore Native and keep rest options to default.

Configuration options

These presets determine how you structure and interact with your data.

- (●) Firestore Native
  Enables Firestore's native server-side, web, and mobile SDKs

  ### Security rules

  These rules provide access control and data validation for web and mobile SDKs.

  - (●) Restrictive
    Deny all reads and writes by default
  - ( ) Open
    Allow anyone to view, edit and delete all data for the next 30 days.

- ( ) Firestore with Datastore compatibility
  Enables Firestore's implementation of Datastore compatibility for server-side SDKs.

5. For the location, choose region of your choice and click on create.

Location

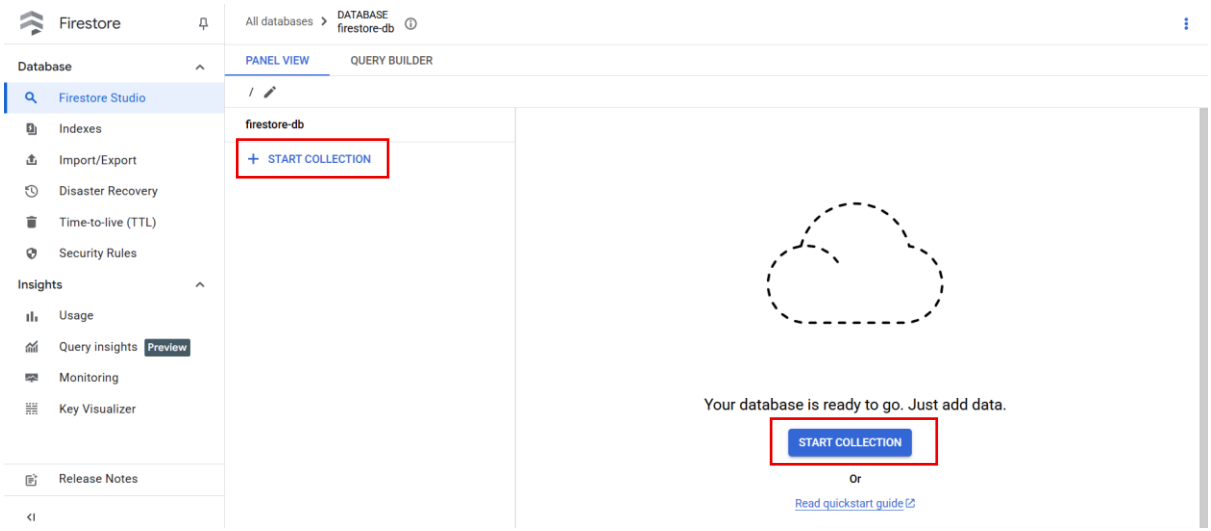
Permanent choice. Determines where your computing resources and data are located. Affects cost, performance and replication. Learn More

Location type

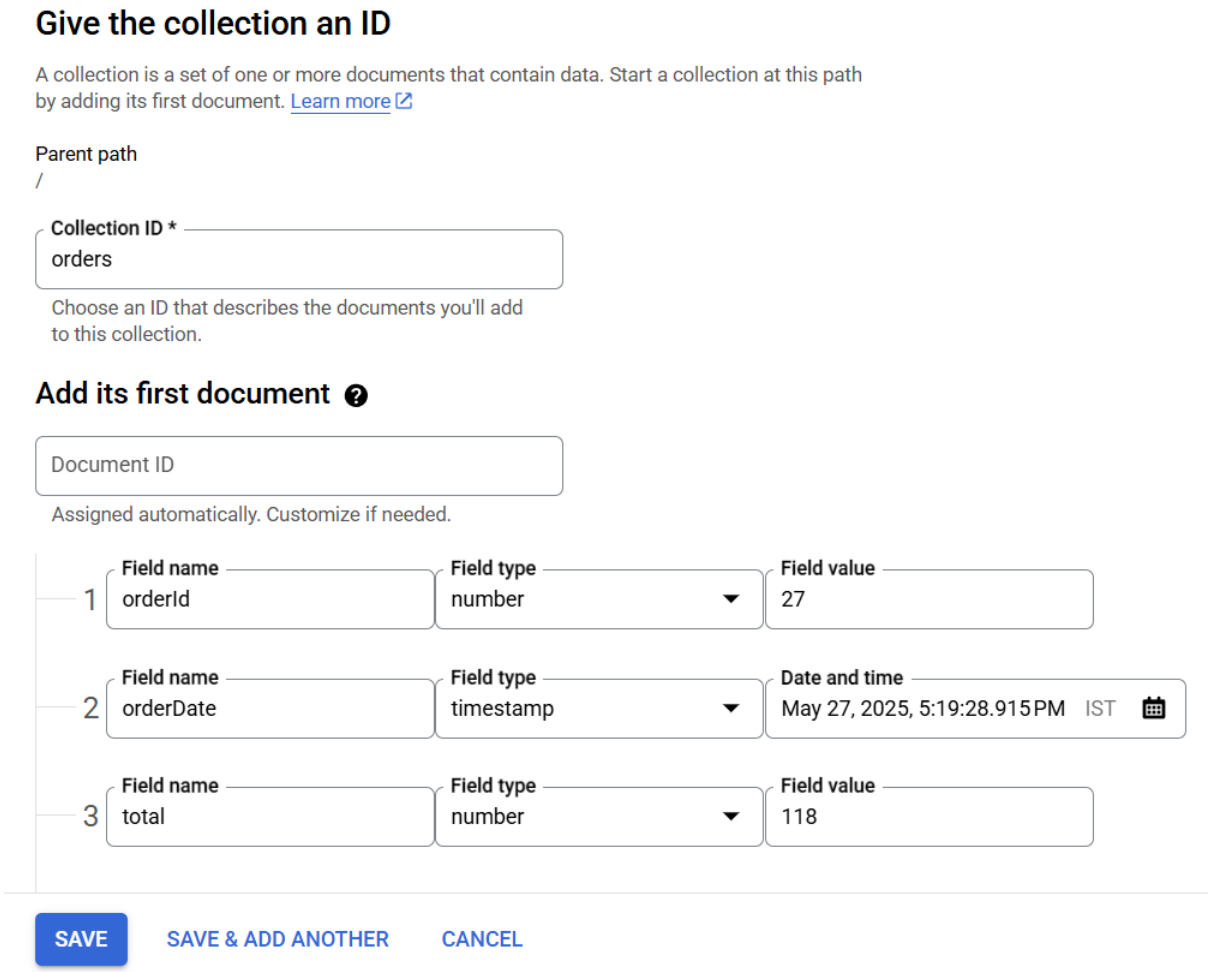
- (●) Region
  99.99% availability SLA at general availability
- ( ) Multi-region
  99.999% availability SLA at general availability, highest availability across the largest area

Region *
us-central1 (Iowa) ▾

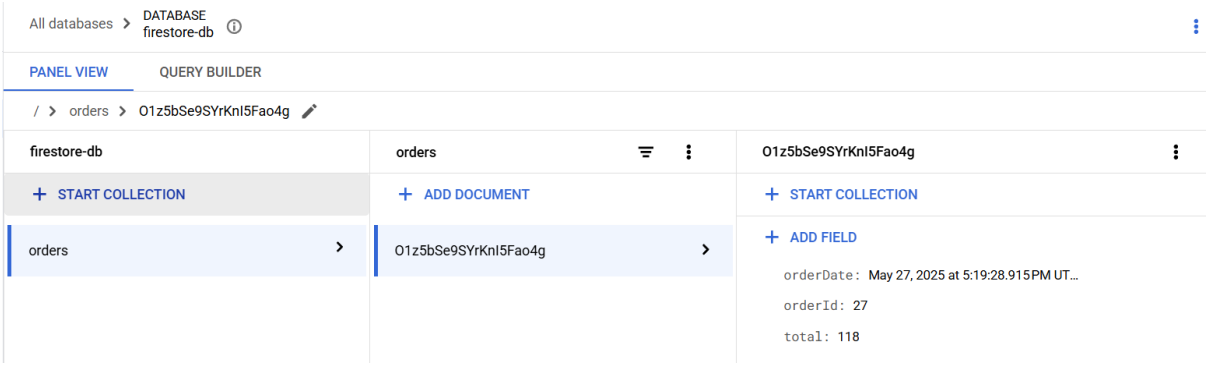
6. Here you can see that our database has been created and now we will insert some data into our database. Click on Start collection.



7. Here, we'll start by giving the collection ID as orders, and then we will give some field names you can choose the same field names as shown below in the snapshot.
8. Do not give the document ID because it will be generated on its own. Click on save.



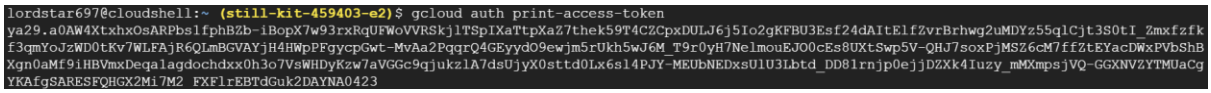
9. In the Firestore database, you can see your orders collection.



10. Now we are going to see how we can access this data using API using the API from the Firestore.
11. But for that, we will need an access token, which will identify us and authorize us to access the database.
12. To create an access token, open the cloud shell and run the command given below.

    gcloud auth print-access-token

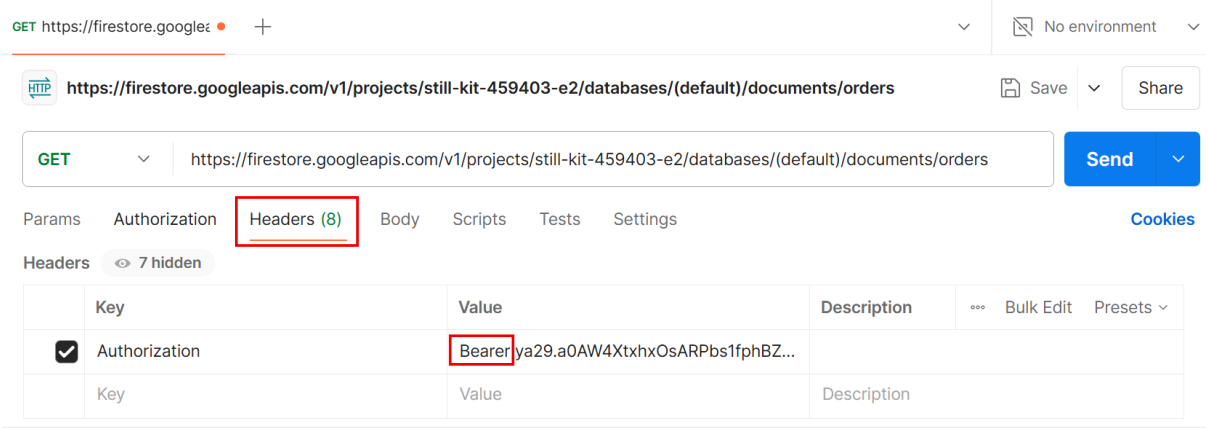
13. Here is the access token generated using the above command.



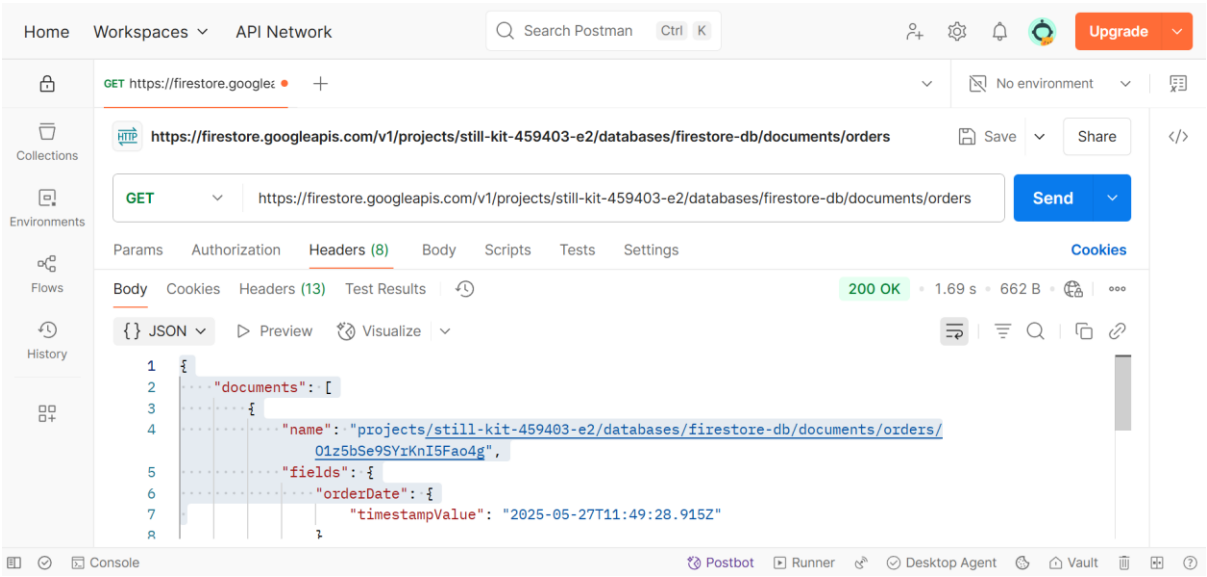
14. Now, since we are going to use the API, the best tool that we have is Postman. Open it.
15. Then, in the Postman, you are going to use the URL given below. Make sure to change the project ID and database name.

    https://firestore.googleapis.com/v1/projects/PROJECT_ID/databases/(default)/documents/orders

16. Now use the above link and paste it using the GET parameter, then choose Headers and in the key write **Authorization** and, in the value, first write **Bearer,** give a space, then paste the access token you get from the cloud shell.
17. Click on Send.



18. Here you can see that we got status 200 OK which our API worked and we got the result as well.



19. In the end, just delete your Firestore Databases.